

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Кафедра вычислительной техники

ОТЧЁТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
ПО ДИСЦИПЛИНЕ «ПРОГРАММИРОВАНИЕ»

Факультет	ЗО	Преподаватель	Дубков Илья
Группа	ДТ-460а		Сергеевич
Студент	Пантюхин Артём		
	Евгеньевич		
Вариант	9		

Новосибирск, 2026 г.

Цель работы

Изучить механизм обработки исключительных ситуаций в объектно-ориентированных программах.

Задание

Добавить в классы и демонстрационную программу обработку исключений при возникновении ошибок: недостатка памяти, выхода за пределы диапазона допустимых значений и т.д. Дополнить демонстрационную программу так, чтобы она демонстрировала обработку исключений.

Исходный код

```
"student_data.h"

#ifndef S3L1_STUDENT_DATA_H
#define S3L1_STUDENT_DATA_H

#include <cstdint>
#include <ostream>

namespace prog_s3 {
    class StudentData {
    public:
        StudentData();
        StudentData(const char* last_name, int age = 0, float average_score =
0.0);

        virtual ~StudentData();
        StudentData(const StudentData &sd);
        StudentData(StudentData &&sd);
        StudentData& operator=(const StudentData& sd);
        StudentData& operator=(StudentData &&sd);

        StudentData& operator++();
        StudentData operator++(int);
        StudentData operator+(float score);
        operator const char*();

        friend StudentData operator-(StudentData& data, float score);

        friend std::ofstream& operator<<(std::ofstream& out, StudentData&
data);

        friend std::ifstream& operator>>(std::ifstream& in, StudentData& data);

        const char* get_last_name() const;
        int get_age() const;
        float get_average_score() const;
        bool set_last_name(const char* name);
        bool set_age(int age);
        bool set_average_score(float score);

        void print_last_name() const;
        void print_age() const;
        void print_average_score() const;
        virtual void print_data() const;

        virtual const char* get_json_string() const;

        StudentData& binary_mode(bool is_binary);

    protected:
        char* _last_name;
        int _age;
        float _average_score;

        float _score_sum;
        std::size_t _score_count;

        bool _ofstream_binary;

        const char _k_default_name[5] = "NULL";
        static void _copy_string(char** dest, const char* src);
        static std::size_t _get_true_size(const char* str);
```

```
static bool _is_valid_name(const char* name);
static bool _is_valid_age(const int age);
static bool _is_valid_average(const float average);

virtual void _save_to_file(std::ofstream& out) const;
virtual void _load_from_file(std::ifstream& in);
virtual void _save_binary(std::ofstream& out) const;
virtual void _load_binary(std::ifstream& in);
};
}

#endif //S3L1_STUDENT_DATA_H
```

"student_data.cpp"

```
#include "student_data.h"
```

```
#include <cstdint>
```

```
#include <cmath>
```

```
#include <cstdio>
```

```
#include <cstring>
```

```
#include <iostream>
```

```
#include <fstream>
```

```
namespace prog_s3 {
```

```
    const char* StudentData::get_last_name() const {  
        return _last_name;  
    }
```

```
    int StudentData::get_age() const {  
        return _age;  
    }
```

```
    float StudentData::get_average_score() const {  
        return _average_score;  
    }
```

```
    bool StudentData::set_last_name(const char* name) {  
        if (!_is_valid_name(name)) {  
            return false;  
        }
```

```
        _copy_string(&_amp;_last_name,name);
```

```
        return true;  
    }
```

```
    bool StudentData::set_age(int age) {  
        if (!_is_valid_age(age)) return false;  
        _age = age;  
        return true;  
    }
```

```
    bool StudentData::set_average_score(float score) {  
        if (!_is_valid_average(score)) return false;  
        _score_count = 1;  
        _score_sum = score;  
        _average_score = score;  
        return true;  
    }
```

```
    StudentData::StudentData():  
        _last_name(NULL),  
        _age(0),  
        _average_score(0.0),  
        _score_count(0),  
        _score_sum(0.0),  
        _ofstream_binary(false)  
    {}
```

```
    StudentData::StudentData(const char* last_name, int age, float average_score):  
    StudentData() {  
        set_last_name(last_name);  
        set_age(age);  
        set_average_score(average_score);  
    }
```

```

}

StudentData::~~StudentData() {
    if (_last_name) {
        delete[] _last_name;
        _last_name = NULL;
    }
}

StudentData::StudentData(const StudentData &sd): StudentData() {
    _age = sd._age;
    _average_score = sd._average_score;
    _copy_string(&_amp;_last_name, sd._last_name);

    _score_count = sd._score_count;
    _score_sum = sd._score_sum;
}

StudentData::StudentData(StudentData &&sd): StudentData() {
    _age = sd._age;
    _average_score = sd._average_score;
    _last_name = sd._last_name;
    sd._last_name = NULL;

    _score_count = sd._score_count;
    _score_sum = sd._score_sum;
}

StudentData& StudentData::operator=(const StudentData& sd) {
    if (&sd == this) return *this;

    _age = sd._age;
    _average_score = sd._average_score;
    _copy_string(&_amp;_last_name, sd._last_name);
    _score_count = sd._score_count;
    _score_sum = sd._score_sum;

    return *this;
}

StudentData& StudentData::operator=(StudentData &&sd) {
    if (&sd == this) return *this;

    _age = sd._age;
    _average_score = sd._average_score;
    if (_last_name) {
        delete [] _last_name;
    }
    _last_name = sd._last_name;
    sd._last_name = NULL;

    _score_count = sd._score_count;
    _score_sum = sd._score_sum;

    return *this;
}

void StudentData::print_last_name() const {
    const char* to_print = _last_name ? _last_name : _k_default_name;
    std::cout << to_print << std::endl;
}

```

```

void StudentData::print_age() const {
    std::cout << _age << std::endl;
}

void StudentData::print_average_score() const {
    char buff[16]; sprintf(buff, "%.2f", _average_score);
    std::cout << buff << std::endl;
}

void StudentData::print_data() const {
    std::cout << "Student: ";
    print_last_name();
    std::cout << "\tAge: ";
    print_age();
    std::cout << "\tAverage Score: ";
    print_average_score();
}

bool StudentData::_is_valid_name(const char* name) {
    if (!name) {
        return false;
    }
    return (bool)(*name != 0);
}

bool StudentData::_is_valid_age(const int age) {
    /*
    понятно, что студент вряд ли будет младенцем или старичком 100+,
    но кто-то, например, может быть зачислен с рождения (как в гарри поттере,

или числиться в базах до смерти,
и мы не хотим вывалиться в ошибку по недосмотру
*/
    return age >= 0 && age <= 150;
}

/*
система оценки не указана, так что просто проверим что это вообще число,
что оно больше нуля,
и далее отдадим всё в руки того, кто будет дёргать за ручки нашего интерфейса.
*/
bool StudentData::_is_valid_average(const float average) {
    if (!std::isfinite(average)) return false;
    return average >= 0.0;
}

void StudentData::_copy_string(char** dest, const char* src) {
    if (!dest) return;
    if (*dest == src) return;

    if (*dest) {
        delete[] *dest;
        *dest = NULL;
    }

    if (src) {
        std::size_t size = _get_true_size(src);
        *dest = new char[size];
        memcpy(*dest, src, size);
        (*dest)[size-1] = 0;
    }
}

```

да)

```

    }

    StudentData& StudentData::operator++() {
        _age++;
        return *this;
    }

    StudentData StudentData::operator++(int) {
        StudentData temp = *this;
        _age++;
        return temp;
    }

    StudentData StudentData::operator+(float score) {
        StudentData new_sd(*this);

        new_sd._score_count+=1;
        new_sd._score_sum+=score;
        new_sd._average_score = new_sd._score_sum/(float)new_sd._score_count;

        return new_sd;
    }

    StudentData operator-(StudentData& lv, float score) {
        /*
        благодаря этой штучке мы не улетим в отрицательные значения
        если на фронте кто-то будет усердно вычитать
        */
        StudentData new_sd(lv);
        float subtract = score <= new_sd._average_score ? score :
new_sd._average_score;
        new_sd._average_score -= subtract;
        new_sd._score_sum = new_sd._average_score*new_sd._score_count;
        return new_sd;
    }

    StudentData::operator const char *() {
        return get_json_string();
    }

    const char * StudentData::get_json_string() const {
        static char buffer[BUFSIZ] = {};
        memset(buffer, 0, BUFSIZ);
        snprintf(buffer, BUFSIZ-1,
            "{ \"_last_name\": \"%s\", \"_age\": \"%d\", \"_average_score\": \"%.2f\",
\"_score_sum\": \"%.2f\", \"_score_count\": \"%ld\" }",
            _last_name ? _last_name : _k_default_name,
            _age,
            _average_score,
            _score_sum,
            _score_count
        );
        return buffer;
    }

    std::size_t StudentData::_get_true_size(const char* str) {
        if (!str) return 0;

        const char* start = str;
        while (*(str++));
        return str-start;
    }

```



```

StudentData& StudentData::binary_mode(bool is_binary) {
    _ofstream_binary = is_binary;
    return *this;
}

std::ofstream& operator<<(std::ofstream& out, StudentData& data) {
    data._save_to_file(out);
    return out;
}

std::ifstream& operator>>(std::ifstream& in, StudentData& data) {
    data._load_from_file(in);
    return in;
}

void StudentData::_save_to_file(std::ofstream& out) const {
    if (_ofstream_binary) {
        _save_binary(out);
    } else {
        out << get_json_string();
    }
}

void StudentData::_load_from_file(std::ifstream& in) {
    _load_binary(in);
}

void StudentData::_save_binary(std::ofstream& out) const {
    std::size_t name_size = StudentData::_get_true_size(_last_name);
    struct save_template {
        int age;
        float average;
        std::size_t score_count;
        float score_sum;
        std::size_t name_size;
    } static_data = {};
    /*
    тут разделили, потому что полезной нагрузки у нас на 28 байт,
    а вот структура получится 32 (выравнивание 8 байт)
    Если игнорировать, получим 4 байта всего что угодно
    между score_sum и name_size, чего не хочется.
    Инициализируем сначала дефолтным конструктором, он забьёт всё нулями
    Потом уже можно.
    */
    static_data = {
        _age,
        _average_score,
        _score_count,
        _score_sum,
        name_size,
    };
    out.write((char*)&static_data, sizeof(static_data));
    if (name_size > 1) {
        out.write(_last_name, name_size);
    }
}

void StudentData::_load_binary(std::ifstream& in) {
    struct save_template {
        int age;

```

```

        float average;
        std::size_t score_count;
        float score_sum;
        std::size_t name_size;
    } static_data = {};
    char* buffer = NULL;

    in.read((char*)&static_data, sizeof(static_data));
    if (in.good() && static_data.name_size > 1) {
        buffer = new char[static_data.name_size];
        in.read(buffer, static_data.name_size);
        buffer[static_data.name_size-1] = 0;
    }
    if (in.good()) {
        if (_last_name) delete [] _last_name;
        _last_name = NULL;
        _age = static_data.age;
        _average_score = static_data.average;
        _score_sum = static_data.score_sum;
        _score_count = static_data.score_count;
    }

    if (buffer) {
        if (in.good()) _copy_string(&_last_name, buffer);
        delete [] buffer;
    }
}
}

```

```

"student_data_budget.h"

#ifndef S3L4_STUDENT_DATA_BUDGET_H
#define S3L4_STUDENT_DATA_BUDGET_H

#include "student_data.h"

#include <fstream>

namespace prog_s3 {
    class StudentDataBudget: public StudentData {
    public:
        StudentDataBudget();
        StudentDataBudget(const char* last_name, int age = 0, float average =
0.0, float grant = 0.0);
        virtual ~StudentDataBudget() = default;

        bool set_grant(float new_grant);
        void print_grant() const;
        virtual void print_data() const override;

        virtual const char* get_json_string() const override;

    protected:
        float _grant;
        bool _is_valid_grant(float value);

        virtual void _save_binary(std::ofstream& out) const override;
        virtual void _load_binary(std::ifstream& in) override;
    }; //class StudentDataBudget
} //namespace prog_s3

#endif //S3L4_STUDENT_BUDGET_H

```

```

"student_data_budget.cpp"

#include "student_data_budget.h"

#include <cmath>
#include <cstring>
#include <iostream>

namespace prog_s3 {
    StudentDataBudget::StudentDataBudget()
        : _grant(0.0) {};

    StudentDataBudget::StudentDataBudget(const char* last_name, int age, float
average, float grant)
        : StudentData(last_name, age, average) {
        _grant = 0.0;
        set_grant(grant);
    }

    bool StudentDataBudget::set_grant(float new_grant) {
        if (!_is_valid_grant(new_grant)) return false;

        _grant = new_grant;
        return true;
    }

    void StudentDataBudget::print_grant() const {
        char buff[32] = {};
        snprintf(buff, 31, "%.2f", _grant);
        std::cout << buff << std::endl;
    }

    void StudentDataBudget::print_data() const {
        StudentData::print_data();
        std::cout << "\tGrant: ";
        print_grant();
    }

    bool StudentDataBudget::_is_valid_grant(float value) {
        if (!std::isfinite(value)) return false;
        return value >= 0.0;
    }

    const char* StudentDataBudget::get_json_string() const {
        static char buffer[BUFSIZ] = {};
        memset(buffer, 0, BUFSIZ);
        snprintf(buffer, BUFSIZ-1,
            "{ \"_last_name\": \"%s\", \"_age\": \"%d\", \"_average_score\": \"%2f\",
\"_score_sum\": \"%2f\", \"_score_count\": \"%ld\", \"_grant\": \"%2f\"}",
            _last_name ? _last_name : _k_default_name,
            _age,
            _average_score,
            _score_sum,
            _score_count,
            _grant
        );
        return buffer;
    }

    void StudentDataBudget::_save_binary(std::ofstream& out) const {
        std::size_t name_size = StudentData::_get_true_size(_last_name);
        struct save_template {

```

```

        int age;
        float average;
        std::size_t score_count;
        float score_sum;
        float grant;
        std::size_t name_size;
    } static_data = {};
    static_data = {
        _age,
        _average_score,
        _score_count,
        _score_sum,
        _grant,
        name_size,
    };
    out.write((char*)&static_data, sizeof(static_data));
    if (name_size > 1) {
        out.write(_last_name, name_size);
    }
}

void StudentDataBudget::_load_binary(std::ifstream& in) {
    struct save_template {
        int age;
        float average;
        std::size_t score_count;
        float score_sum;
        float grant;
        std::size_t name_size;
    } static_data = {};
    char* buffer = NULL;

    in.read((char*)&static_data, sizeof(static_data));
    if (in.good() && static_data.name_size > 1) {
        buffer = new char[static_data.name_size];
        in.read(buffer, static_data.name_size);
        buffer[static_data.name_size-1] = 0;
    }

    if (in.good()) {
        if (_last_name) delete [] _last_name;
        _last_name = NULL;
        _age = static_data.age;
        _average_score = static_data.average;
        _score_sum = static_data.score_sum;
        _score_count = static_data.score_count;
        _grant = static_data.grant;
    }

    if (buffer) {
        if (in.good()) _copy_string(&_last_name, buffer);
        delete [] buffer;
    }
}
} //namespace prog_s3

```

"student_data_debt.h"

```

#ifndef S3L4_STUDENT_DATA_DEBT_H
#define S3L4_STUDENT_DATA_DEBT_H

```

```

#include "student_data.h"

#include <fstream>

namespace prog_s3 {
    class StudentDataDebt: public StudentData {
    public:
        StudentDataDebt();
        StudentDataDebt(const char* last_name, int age = 0, float average =
0.0, int debts = 0);
        virtual ~StudentDataDebt() = default;

        bool set_debts(int count);
        void close_debt(int amount = 1);
        void add_debt(int amount = 1);

        void print_debt() const;
        virtual void print_data() const override;
        virtual const char* get_json_string() const override;

    protected:
        int _debts;
        bool _is_valid_debt(int value);

        virtual void _save_binary(std::ofstream& out) const override;
        virtual void _load_binary(std::ifstream& in) override;

    }; //class StudentDataDebt
} //namespace prog_s3

#endif

```

"student_data_debt.cpp"

```
#include "student_data_debt.h"
```

```
#include <cstring>
```

```
#include <iostream>
```

```
namespace prog_s3 {
```

```
    StudentDataDebt::StudentDataDebt(): _debts(0) {}
```

```
    StudentDataDebt::StudentDataDebt(  
        const char* last_name, int age, float average, int debts)  
        :StudentData(last_name, age, average) {  
        _debts = 0;  
        set_debts(debts);  
    }  
}
```

```
bool StudentDataDebt::set_debts(int count) {  
    if (!_is_valid_debt(count)) return false;  
    _debts = count;  
    return true;  
}
```

```
void StudentDataDebt::close_debt(int amount) {  
    if (amount > 0) {  
        _debts -= amount > _debts ? _debts : amount;  
    }  
}
```

```
void StudentDataDebt::add_debt(int amount) {  
    if (amount > 0) {  
        _debts += amount;  
    }  
}
```

```
void StudentDataDebt::print_debt() const {  
    std::cout << _debts << std::endl;  
}
```

```
void StudentDataDebt::print_data() const {  
    StudentData::print_data();  
    std::cout << "\tDebts: ";  
    print_debt();  
}
```

```
bool StudentDataDebt::_is_valid_debt(int value) {  
    return value >= 0;  
}
```

```
const char* StudentDataDebt::get_json_string() const {  
    static char buffer[BUFSIZ] = {};  
    memset(buffer, 0, BUFSIZ);  
    snprintf(buffer, BUFSIZ-1,  
        "{ \"_last_name\": \"%s\", \"_age\": \"%d\", \"_average_score\": \"%.2f\",  
        \"_score_sum\": \"%.2f\", \"_score_count\": \"%ld\", \"_debts\": \"%d\" }",  
        _last_name ? _last_name : _k_default_name,  
        _age,  
        _average_score,  
        _score_sum,  
        _score_count,  
        _debts  
    );  
    return buffer;  
}
```

```

    }

    void StudentDataDebt::_save_binary(std::ofstream& out) const {
        std::size_t name_size = StudentData::_get_true_size(_last_name);
        struct save_template {
            int age;
            float average;
            std::size_t score_count;
            float score_sum;
            int debts;
            std::size_t name_size;
        } static_data = {};
        static_data = {
            _age,
            _average_score,
            _score_count,
            _score_sum,
            _debts,
            name_size,
        };
        out.write((char*)&static_data, sizeof(static_data));
        if (name_size > 1) {
            out.write(_last_name, name_size);
        }
    }

    void StudentDataDebt::_load_binary(std::ifstream& in) {
        struct save_template {
            int age;
            float average;
            std::size_t score_count;
            float score_sum;
            int debts;
            std::size_t name_size;
        } static_data = {};
        char* buffer = NULL;

        in.read((char*)&static_data, sizeof(static_data));

        if (in.good() && static_data.name_size > 1) {
            buffer = new char[static_data.name_size];
            in.read(buffer, static_data.name_size);
            buffer[static_data.name_size-1] = 0;
        }

        if (in.good()) {
            if (_last_name) delete [] _last_name;
            _last_name = NULL;
            _age = static_data.age;
            _average_score = static_data.average;
            _score_sum = static_data.score_sum;
            _score_count = static_data.score_count;
            _debts = static_data.debts;
        }

        if (buffer) {
            if (in.good()) _copy_string(&_last_name, buffer);
            delete [] buffer;
        }
    }

} //namespace prog_s3

```


"data_list.h"

```
#ifndef S3L5_DATA_LIST_H
#define S3L5_DATA_LIST_H
```

```
#include "student_data.h"
#include "student_data_budget.h"
#include "student_data_debt.h"
```

```
namespace prog_s3 {
    class DataList {
    public:
        DataList();
        DataList(DataList&&);
        ~DataList();

        void clear();

        void push_front(StudentData& item);
        void push_back(StudentData& item);
        bool insert_at(StudentData& item, std::size_t pos = 0);

        void push_front(StudentDataDebt& item);
        void push_back(StudentDataDebt& item);
        bool insert_at(StudentDataDebt& item, std::size_t pos = 0);

        void push_front(StudentDataBudget& item);
        void push_back(StudentDataBudget& item);
        bool insert_at(StudentDataBudget& item, std::size_t pos = 0);

        bool pop_front();
        bool pop_back();

        bool erase(StudentData* element);
        bool remove_at(std::size_t pos);

        StudentData* get(std::size_t pos);
        StudentData* find(const char* name, std::size_t from = 0);

        std::size_t count(const char* name, std::size_t from = 0);

        std::size_t size();
        bool is_empty();

        void print_structure();

    protected:
        class DataNode {
        public:
            DataNode();
            DataNode(StudentData *value, DataNode *next = NULL);
            ~DataNode();
            StudentData* value;
            DataNode* next;

        }; //class DataNode
    private:
        bool _insert_ptr(StudentData* value, std::size_t pos);
        void _push_back_ptr(StudentData* value);
        void _push_front_ptr(StudentData* value);
        void _delete_node(DataNode** node, DataNode* prev = NULL);
        DataNode* _root;
    };
}
```

```
        std::size_t _size;
    };//class DataList
} //namespace prog_s3

#endif //S3_DATA_LIST_H
```

"data_list.cpp"

```
#include "data_list.h"
#include "student_data.h"
#include "student_data_budget.h"
#include "student_data_debt.h"

#include <cstddef>
#include <cstring>
#include <iostream>
#include <stdexcept>

namespace prog_s3 {
    DataList::DataList(): _root(NULL), _size(0) {};
    DataList::DataList(DataList&& other) {
        clear();
        this->_root = other._root;
        this->_size = other._size;
        other._root = NULL;
        other._size = 0;
    }

    DataList::~DataList() {
        clear();
    }

    void DataList::clear() {
        while (pop_front());
        _size = 0;
        _root = NULL;
    }

    void DataList::push_front(StudentData& item) {
        _push_front_ptr(new StudentData(item));
    }

    void DataList::push_back(StudentData& item) {
        _push_back_ptr(new StudentData(item));
    }

    bool DataList::insert_at(StudentData& item, std::size_t pos) {
        StudentData* new_ptr = new StudentData(item);
        bool is_ok = _insert_ptr(new_ptr, pos);
        if (!is_ok) {
            delete new_ptr;
        }
        return is_ok;
    }

    void DataList::push_front(StudentDataDebt& item) {
        _push_front_ptr((StudentData*) new StudentDataDebt(item));
    }

    void DataList::push_back(StudentDataDebt& item) {
        _push_back_ptr((StudentData*) new StudentDataDebt(item));
    }

    bool DataList::insert_at(StudentDataDebt& item, std::size_t pos) {
        StudentData* new_ptr = (StudentData*) new StudentDataDebt(item);
        bool is_ok = _insert_ptr(new_ptr, pos);
        if (!is_ok) {
            delete new_ptr;
        }
    }
}
```

```

        return is_ok;
    }

void DataList::push_front(StudentDataBudget& item) {
    _push_front_ptr((StudentData*) new StudentDataBudget(item));
}

void DataList::push_back(StudentDataBudget& item) {
    _push_back_ptr((StudentData*) new StudentDataBudget(item));
}

bool DataList::insert_at(StudentDataBudget& item, std::size_t pos) {
    StudentData* new_ptr = (StudentData*) new StudentDataBudget(item);
    bool is_ok = _insert_ptr(new_ptr, pos);
    if (!is_ok) {
        delete new_ptr;
    }
    return is_ok;
}

bool DataList::pop_front() {
    if (!_root || is_empty()) return false;
    _delete_node(&_root, NULL);

    return true;
}

bool DataList::pop_back() {
    if (!_root || is_empty()) return false;

    DataNode* ptr = _root;
    DataNode* prev = NULL;
    while (ptr->next) {
        prev = ptr;
        ptr=ptr->next;
    }
    _delete_node(&ptr, prev);
    return true;
}

bool DataList::erase(StudentData* element) {
    if (!_root || is_empty()) return false;

    DataNode* ptr = _root;
    DataNode* prev = NULL;
    while (ptr && ptr->value != element) {
        prev = ptr;
        ptr = ptr->next;
    }

    if (ptr) {
        _delete_node(&ptr, prev);
        return true;
    }
    return false;
}

bool DataList::remove_at(std::size_t pos) {
    if (is_empty() || pos >= _size) throw std::out_of_range("Error: pos >=
_size");

    DataNode* delptr = _root;
    DataNode* prev = NULL;

```

```

        for(std::size_t i = 0; i < pos; i++) {
            prev = delptr;
            delptr = delptr->next;
        }
        _delete_node(&delptr, prev);
        return true;
    }

    StudentData* DataList::get(std::size_t pos) {
        if (is_empty() || pos >= _size) throw std::out_of_range("Error: pos >=
_size");

        DataNode* ptr = _root;
        for (std::size_t i = 0; i < pos && ptr; i++) {
            ptr = ptr->next;
        }
        StudentData* retval = ptr ? ptr->value : NULL;
        return retval;
    }

    StudentData* DataList::find(const char* name, std::size_t from) {
        if (is_empty() || from >= _size) throw std::out_of_range("Error: from >=
_size");

        StudentData* retval = NULL;
        DataNode* ptr = _root;
        for (std::size_t i = from; i < _size && !retval; i++) {
            if (strcmp(ptr->value->get_last_name(), name) == 0) {
                retval = ptr->value;
            }
            ptr = ptr->next;
        }
        return retval;
    }

    std::size_t DataList::count(const char* name, std::size_t from) {
        if (is_empty() || from >= _size) throw std::out_of_range("Error: from >=
_size");

        std::size_t amount = 0;
        DataNode* ptr = _root;
        for (std::size_t i = from; i < _size; i++) {
            amount += (strcmp(ptr->value->get_last_name(), name) == 0);
            ptr = ptr->next;
        }
        return amount;
    }

    std::size_t DataList::size() {
        return _size;
    }

    bool DataList::is_empty() {
        return _size == 0;
    }

    bool DataList::_insert_ptr(StudentData* value, std::size_t pos) {
        if (pos > _size) throw std::out_of_range("Error: pos>_size");
        if (!value) throw std::invalid_argument("value == NULL");
        DataNode* ptr = _root;
        DataNode* prev = NULL;
        for (std::size_t i = 0; i < pos; i++) {
            prev = ptr;
            ptr = ptr->next;
        }

```

```

        if (!prev) {
            _root = new DataNode(value, ptr);
        } else {
            prev->next = new DataNode(value, ptr);
        }
        _size++;
        return true;
    }

    void DataList::_push_back_ptr(StudentData* value) {
        _insert_ptr(value, _size);
    }

    void DataList::_push_front_ptr(StudentData* value) {
        _insert_ptr(value, 0);
    }

    void DataList::_delete_node(DataNode** node, DataNode* prev) {
        if (is_empty() || !node) return;
        if (!*node) return;

        DataNode* next = (*node)->next;
        delete *node;
        if (prev == NULL || prev == *node) {
            *node = next;
        } else {
            prev->next = next;
        }
        _size -= 1;
    }

    DataList::DataNode::DataNode():DataNode(NULL, NULL) {};
    DataList::DataNode::DataNode(StudentData *value, DataNode *next)
        : value(value), next(next) {};
    DataList::DataNode::~~DataNode() {
        if (value) delete value;
    }

    void DataList::print_structure() {
        DataNode* ptr = _root;
        std::size_t count = 0;
        //уходим от стандарта, на msvc может и сломаться
        while (ptr) {
            for (std::size_t i = 0; i < count*2; i++) {
                std::cout << " ";
            }
            std::cout << count << ": " << typeid(ptr->value).name() << "(" <<
typeid(*(ptr->value)).name() << ")" << std::endl;
            ptr = ptr->next;
            count++;
        }
    }
}

```

"main.cpp"

```
#include "student_data.h"
#include "student_data_budget.h"
#include "student_data_debt.h"
#include "data_list.h"

#include <fstream>
#include <iostream>
#include <new>
#include <stdexcept>

void print_title(const char* title) {
    std::cout << "\033[32m|\\" << title << " |\\|\033[0m" << std::endl;
}

void print_closing(const char* title) {
    std::cout << "\033[32m|^| " << title << " |^|\033[0m" << std::endl;
}

void print_valid_example(prog_s3::StudentData& data) {
    print_title("Setters example (valid)");
    std::cout << "\tSET: Last name: Zaharov" << std::endl;
    std::cout << "\tSET: Age: 33" << std::endl;
    std::cout << "\tSET: Average score: 68.8" << std::endl << std::endl;

    data.set_last_name("Zaharov");
    data.set_age(33);
    data.set_average_score(68.8);
    data.print_data();

    print_closing("Setters example (valid) end");
}

void print_invalid_example(prog_s3::StudentData& data) {
    print_title("Setters example (invalid)");
    std::cout << "\tSET: Last name: \"\"" << std::endl;
    std::cout << "\tSET: Age: -6" << std::endl;
    std::cout << "\tSET: Average score: -88.5" << std::endl << std::endl;

    data.set_last_name("");
    data.set_age(-6);
    data.set_average_score(-88.5);
    data.print_data();
    print_closing("Setters example (invalid) end");
}

prog_s3::StudentData setup_copy() {
    prog_s3::StudentData source("Ryazantsev", 24, 96.3);
    prog_s3::StudentData dest(source);
    return dest;
}

void print_simple_part() {
    /*
    тут не будет try/catch,
    поскольку дефолтный конструктор память не выделяет,
    а больше причин ругаться у него нет
    */
    prog_s3::StudentData student_simple{};
    print_title("Simple constructor");
    student_simple.print_data();
}
```

```

    print_closing("Simple constructor end");
    print_valid_example(student_simple);
    print_invalid_example(student_simple);
}

void print_various_constructors_part() {
    print_title("Full constructor");
    try {
        prog_s3::StudentData student_all_fields("Ryazantsev", 24, 96.3);
        student_all_fields.print_data();
    } catch (std::bad_alloc ex) {
        std::cout << "Full constructor: bad allocation" << std::endl;
        std::cerr << ex.what() << std::endl;
    }
    print_closing("Full constructor end");

    print_title("Full constructor (invalid)");
    try {
        prog_s3::StudentData student_incorrect("", -444, -3.2);
        student_incorrect.print_data();
    } catch (std::bad_alloc ex) {
        std::cout << "Invalid constructor: bad allocation" << std::endl;
        std::cerr << ex.what() << std::endl;
    }
    print_closing("Full constructor (invalid) end");

    print_title("Copy constructor (Ryazantsev to new)");
    /*
    Чтобы показать копирование значения, а не поинтера,
    вызываю отдельную функцию, по истечении которой исходник удалится.
    */
    try {
        prog_s3::StudentData student_copied = setup_copy();
        std::cout << "Original initialized out of scope and is deleted by this
point" << std::endl;
        student_copied.print_data();
    } catch (std::bad_alloc ex) {
        std::cout << "Copy constructor: bad allocation" << std::endl;
        std::cerr << ex.what() << std::endl;
    }
    print_closing("Copy constructor end");

    print_title("Move constructor (From Ryazantsev)");
    try {
        prog_s3::StudentData student_source("Ryazantsev", 24, 96.3);
        prog_s3::StudentData student_moved(std::move(student_source));
        std::cout << "Destination: " << std::endl;
        student_moved.print_data();
        std::cout << "Source: (will be deleted after function's return)" <<
std::endl;
        student_source.print_data();
    } catch (std::bad_alloc ex) {
        std::cout << "Move constructor: bad allocation" << std::endl;
        std::cerr << ex.what() << std::endl;
    }
    print_closing("Move constructor end");
}

void print_getters() {
    print_title("Getters example");
}

```



```

    try {
        prog_s3::StudentData student("Ryazantsev", 24, 96.3);
        student.print_data();
        std::cout << std::endl;
        std::cout << "student.get_last_name(): " << student.get_last_name() <<
std::endl;
        std::cout << "student.get_age(): " << student.get_age() << std::endl;
        std::cout << "student.get_average_score(): " << student.get_average_score()
<< std::endl;
    } catch (std::bad_alloc ex) {
        std::cout << "print_getters() - bad allocation" << std::endl;
        std::cerr << ex.what() << std::endl;
    }
    print_closing("Getters example end");
}

void print_prints() {
    print_title("Print functions example");
    try {
        prog_s3::StudentData student("Ryazantsev", 24, 96.3);
        std::cout << "student.print_data():" << std::endl;
        student.print_data();
        std::cout << "student.print_last_name():" << std::endl;
        student.print_last_name();
        std::cout << "student.print_age():" << std::endl;
        student.print_age();
        std::cout << "student.print_average_score():" << std::endl;
        student.print_average_score();
    } catch (std::bad_alloc ex) {
        std::cout << "print_prints() - bad allocation" << std::endl;
        std::cerr << ex.what() << std::endl;
    }
    print_closing("Print functions end");
}

void print_move_assignment() {
    print_title("Move assignment example");
    //неохота совать всё в большой try/catch, сделаю pointers ввне скопа
    prog_s3::StudentData* s1ptr = NULL;
    prog_s3::StudentData* s2ptr = NULL;
    try {
        s1ptr = new prog_s3::StudentData("Ryazantsev", 24, 96.3);
        s2ptr = new prog_s3::StudentData("Zaharov", 33, 68.8);
    } catch (std::bad_alloc ex) {
        std::cout << "print_move_assignment() - bad alloc"
        << " during initialization" << std::endl;
        std::cerr << ex.what() << std::endl;
        if (s1ptr) delete s1ptr;
        if (s2ptr) delete s2ptr;
        print_closing("Move assignment example end");
        return;
    }
    char buff[16];

    std::cout << "Start values" << std::endl;
    std::cout << "s1:" << std::endl;
    s1ptr->print_data();
    std::cout << "s2:" << std::endl;
    s2ptr->print_data();

    sprintf(buff, "%p", s1ptr->get_last_name());
    std::cout << std::endl << "s1 name pointer: " << buff << std::endl;
}

```

```

    sprintf(buff, "%p", s2ptr->get_last_name());
    std::cout << "s2 name pointer: " << buff << std::endl;

    std::cout << "s1 = std::move(s2)" << std::endl;
    try {
        *s1ptr = std::move(*s2ptr);
    } catch (std::bad_alloc ex) {
        std::cout << "print_move_assignment() - bad alloc"
            << " during assignment" << std::endl;
        std::cerr << ex.what() << std::endl;
        if (s1ptr) delete s1ptr;
        if (s2ptr) delete s2ptr;
        print_closing("Move assignment example end");
        return;
    }

    std::cout << "s1:" << std::endl;
    s1ptr->print_data();
    std::cout << "s2:" << std::endl;
    s2ptr->print_data();

    sprintf(buff, "%p", s1ptr->get_last_name());
    std::cout << std::endl << "s1 name pointer: " << buff << std::endl;
    sprintf(buff, "%p", s2ptr->get_last_name());
    std::cout << "s2 name pointer: " << buff << std::endl;

    if (s1ptr) delete s1ptr;
    if (s2ptr) delete s2ptr;

    print_closing("Move assignment example end");
}

void print_copy_assignment() {
    print_title("Copy assignment example");
    prog_s3::StudentData* s1ptr = NULL;
    prog_s3::StudentData* s2ptr = NULL;
    try {
        s1ptr = new prog_s3::StudentData("Ryazantsev", 24, 96.3);
        s2ptr = new prog_s3::StudentData("Zaharov", 33, 68.8);
    } catch (std::bad_alloc ex) {
        std::cout << "print_copy_assignment() - bad alloc"
            << " during initialization" << std::endl;
        std::cerr << ex.what() << std::endl;
        if (s1ptr) delete s1ptr;
        if (s2ptr) delete s2ptr;
        print_closing("Copy assignment example end");
        return;
    }

    std::cout << "Start values" << std::endl;
    std::cout << "s1:" << std::endl;
    s1ptr->print_data();
    std::cout << "s2:" << std::endl;
    s2ptr->print_data();

    std::cout << "s1 = s2" << std::endl;
    try {
        *s1ptr = *s2ptr;
    } catch (std::bad_alloc ex) {
        std::cout << "print_copy_assignment() - bad alloc"
            << " during assignment" << std::endl;
        std::cerr << ex.what() << std::endl;
    }
}

```

```

        if (s1ptr) delete s1ptr;
        if (s2ptr) delete s2ptr;
        print_closing("Copy assignment example end");
        return;
    }

    std::cout << "s1:" << std::endl;
    s1ptr->print_data();
    std::cout << "s2:" << std::endl;
    s2ptr->print_data();

    char buff[16];
    sprintf(buff, "%p", s1ptr->get_last_name());
    std::cout << std::endl << "s1 name pointer: " << buff << std::endl;
    sprintf(buff, "%p", s2ptr->get_last_name());
    std::cout << "s2 name pointer: " << buff << std::endl;

    if (s1ptr) delete s1ptr;
    if (s2ptr) delete s2ptr;

    print_closing("Copy assignment example end");
}

void print_assignment() {
    print_move_assignment();
    print_copy_assignment();
}

void print_increment() {
    print_title("Increments overloading example");
    try {
        prog_s3::StudentData student("Ryazantsev", 24, 96.3);
        std::cout << "Start values: " << std::endl;
        student.print_data();
        std::cout << "\n(++student).print_data()" << std::endl;
        (++student).print_data();
        std::cout << "\n(student++).print_data()" << std::endl;
        (student++).print_data();
        std::cout << "\nEnd values:" << std::endl;
        student.print_data();
    } catch (std::bad_alloc ex) {
        std::cout << "print_increment() - bad alloc" << std::endl;
        std::cerr << ex.what() << std::endl;
    }
    print_closing("Increments overloading end");
}

void print_addition() {
    print_title("Addition overloading example");
    try {
        prog_s3::StudentData student("Ryazantsev", 24, 96.3);
        std::cout << "Start values:" << std::endl;
        student.print_data();
        std::cout << "\nstudent = student+0.0" << std::endl;
        std::cout << "Expected: average score = (96.3+0.0)/2 = 48.15" << std::endl;
        student = student+0.0;
        student.print_data();
        std::cout << "\nstudent = student+55.4" << std::endl;
        std::cout << "Expected: average score = (96.3+55.4)/3 ~= 50.57" <<
std::endl;
        student = student+55.4;

```

```

        student.print_data();
    } catch (std::bad_alloc ex) {
        std::cout << "print_addition() - bad alloc" << std::endl;
        std::cerr << ex.what() << std::endl;
    }
    print_closing("Addition example end");
}

void print_subtraction() {
    print_title("Subtraction overloading example");
    try {
        prog_s3::StudentData student("Ryazantsev", 24, 96.3);
        student = (student + 0.0) + 55.4;
        std::cout << "Start values:" << std::endl;
        student.print_data();

        std::cout << "\nstudent = student-0.0" << std::endl;
        std::cout << "Expected: average score = 50.57" << std::endl;
        student = student - 0.0;
        student.print_data();

        std::cout << "\nstudent = student - 15.52" << std::endl;
        std::cout << "Expected: average score = 35.05" << std::endl;
        student = student - 15.52;
        student.print_data();

        std::cout << "\nstudent = student - 88.3" << std::endl;
        std::cout << "Expected: average score = 0.0" << std::endl;
        student = student - 88.3;
        student.print_data();
    } catch (std::bad_alloc ex) {
        std::cout << "print_subtraction() - bad alloc" << std::endl;
        std::cerr << ex.what() << std::endl;
    }

    print_closing("Subtraction overloading end");
}

void print_conversion() {
    print_title("Implicit conversion overloading example");
    try {
        prog_s3::StudentData student("Ryazantsev", 24, 96.3);
        std::cout << "Start values" << std::endl;
        student.print_data();

        std::cout << "std::cout << student << std::endl;" << std::endl;
        std::cout << "Expected: Implicit conversion of student object into JSON
string" << std::endl;
        std::cout << student << std::endl;
    } catch (std::bad_alloc ex) {
        std::cout << "print_conversion() - bad alloc" << std::endl;
        std::cerr << ex.what() << std::endl;
    }
    print_closing("Implicit conversion end");
}

void print_files() {
    print_title("Outputting to files: data.txt, data.bin");
    prog_s3::StudentData* sptr = NULL;
    try {
        sptr = new prog_s3::StudentData("Ryazantsev", 24, 96.3);
    } catch (std::bad_alloc ex) {

```

```

        std::cout << "print_files() - bad alloc on init" << std::endl;
        std::cerr << ex.what() << std::endl;
        if (sptr) delete sptr;
        return;
    }

    //iostream и производные по умолчанию не бросают исключения
    std::ofstream text("data.txt", std::ios::out);
    if (text.good()) {
        text << *sptr;
        text.close();
    } else {
        std::cerr << "Cannot open file \"data.txt\" in write mode" << std::endl;
    }

    std::ofstream bin("data.bin", std::ofstream::out | std::ofstream::binary);
    if (bin.good()) {
        bin << sptr->binary_mode(true);
        bin.close();
    } else {
        std::cerr << "Cannot open file \"data.bin\" in write mode" << std::endl;
    }

    if (sptr) delete sptr;
    print_closing("Outputting to files end");
}

void load_from_binary() {
    print_title("Loading from binary: data.bin");
    prog_s3::StudentData new_student{};
    std::cout << "Before load: " << std::endl;
    new_student.print_data();

    //iostream и производные по умолчанию не бросают исключения
    std::ifstream bin("data.bin", std::ifstream::in | std::ifstream::binary);
    if (bin.good()) {
        try {
            bin >> new_student;
        } catch (std::bad_alloc ex) {
            std::cout << "load_from_binary() - bad alloc on loading" << std::endl;
            std::cerr << ex.what() << std::endl;
        }
        bin.close();
        std::cout << "After load: " << std::endl;
        new_student.print_data();
    } else {
        std::cerr << "Cannot open file \"data.bin\" in read mode. Does it exist?"
<< std::endl;
    }
    print_closing("Loading from binary end");
}

void print_file_io() {
    print_files();
    load_from_binary();
}

void print_constructors_budget() {
    print_title("Derived:Budget, constructors");
    /*
        Это ужасная практика и мне стыдно,
        но дальнейшие конструкторы зависят от текущих,

```

```

        а переписывать - долго, и времени нет.
    */
    try {
        std::cout << "Simple constructor" << std::endl;
        prog_s3::StudentDataBudget simple{};
        simple.print_data();

        std::cout << std::endl << "Full constructor" << std::endl;
        prog_s3::StudentDataBudget defined("Ryazantsev", 24, 96.3, 3);
        defined.print_data();

        std::cout << std::endl << "Copy constructor (default)" << std::endl;
        std::cout << "Source object:" << std::endl;
        defined.print_data();
        std::cout << "New object:" << std::endl;
        simple = prog_s3::StudentDataBudget(defined);
        simple.print_data();
        char buff[16];
        sprintf(buff, "%p", defined.get_last_name());
        std::cout << "Source name pointer: " << buff << std::endl;
        sprintf(buff, "%p", simple.get_last_name());
        std::cout << "New name pointer: " << buff << std::endl;

        std::cout << std::endl << "Copy constructor (base(derived))" << std::endl;
        std::cout << "Source object:" << std::endl;
        defined.print_data();
        std::cout << "New object:" << std::endl;
        prog_s3::StudentData base(defined);
        base.print_data();
        sprintf(buff, "%p", defined.get_last_name());
        std::cout << "Source name pointer: " << buff << std::endl;
        sprintf(buff, "%p", base.get_last_name());
        std::cout << "New name pointer: " << buff << std::endl;
    } catch (std::bad_alloc ex) {
        std::cout << "print_constructors_budget() - bad alloc" << std::endl;
        std::cerr << ex.what() << std::endl;
    }

    print_closing("Derived:Budget, constructors end");
}

void print_constructors_debt(){
    print_title("Derived:Debt, constructors");
    /*
        Это ужасная практика и мне стыдно,
        но дальнейшие конструкторы зависят от текущих,
        а переписывать - долго, и времени нет.
    */
    try {
        std::cout << "Simple constructor" << std::endl;
        prog_s3::StudentDataDebt simple{};
        simple.print_data();

        std::cout << std::endl << "Full constructor" << std::endl;
        prog_s3::StudentDataDebt defined("Ryazantsev", 24, 96.3, 3);
        defined.print_data();

        std::cout << std::endl << "Copy constructor (default)" << std::endl;
        std::cout << "Source object:" << std::endl;
        defined.print_data();
        std::cout << "New object:" << std::endl;
        simple = prog_s3::StudentDataDebt(defined);
    }
}

```

```

    simple.print_data();
    char buff[16];
    sprintf(buff, "%p", defined.get_last_name());
    std::cout << "Source name pointer: " << buff << std::endl;
    sprintf(buff, "%p", simple.get_last_name());
    std::cout << "New name pointer: " << buff << std::endl;

    std::cout << std::endl << "Copy constructor (base(derived))" << std::endl;
    std::cout << "Source object:" << std::endl;
    defined.print_data();
    std::cout << "New object:" << std::endl;
    prog_s3::StudentData base(defined);
    base.print_data();
    sprintf(buff, "%p", defined.get_last_name());
    std::cout << "Source name pointer: " << buff << std::endl;
    sprintf(buff, "%p", base.get_last_name());
    std::cout << "New name pointer: " << buff << std::endl;
} catch (std::bad_alloc ex) {
    std::cout << "print_constructors_debt() - bad alloc" << std::endl;
    std::cerr << ex.what() << std::endl;
}

    print_closing("Derived:Debt, constructors end");
}

void print_derived_constructors() {
    print_constructors_debt();
    print_constructors_budget();
}

void print_virtuals_debt() {
    print_title("Derived:Debt, virtual example");
    try {
        prog_s3::StudentData parent("Ryazantsev", 24, 96.3);
        prog_s3::StudentDataDebt derived("Ryazantsev", 24, 96.3, 3);
        std::cout << "parent.get_json_string(): " << std::endl <<
parent.get_json_string() << std::endl;
        std::cout << "derived.get_json_string(): " << std::endl <<
derived.get_json_string() << std::endl;
        std::cout << "((ParentClass*) &derived)->get_json_string() [[OVERRIDDEN]]:"
<< std::endl << ((prog_s3::StudentData*) &derived)->get_json_string() << std::endl;
    } catch (std::bad_alloc ex) {
        std::cout << "print_virtuals_debt() - bad alloc on init" << std::endl;
        std::cerr << ex.what() << std::endl;
    }

    print_closing("Derived:Debt, virtual example end");
}

void print_virtuals_budget() {
    print_title("Derived:Budget, virtual example");
    try {
        prog_s3::StudentData parent("Ryazantsev", 24, 96.3);
        prog_s3::StudentDataBudget derived("Ryazantsev", 24, 96.3, 330.53);
        std::cout << "parent.get_json_string(): " << std::endl <<
parent.get_json_string() << std::endl;
        std::cout << "derived.get_json_string(): " << std::endl <<
derived.get_json_string() << std::endl;
        std::cout << "((ParentClass*) &derived)->get_json_string() [[OVERRIDDEN]]:"
<< std::endl << ((prog_s3::StudentData*) &derived)->get_json_string() << std::endl;
    } catch (std::bad_alloc ex) {
        std::cout << "print_virtuals_budget() - bad alloc on init" << std::endl;
    }
}

```

```

        std::cerr << ex.what() << std::endl;
    }
    print_closing("Derived:Budget, virtual example end");
}

void print_derived_virtuals() {
    print_virtuals_debt();
    print_virtuals_budget();
}

void print_polymorphic() {
    print_title("Simple StudentData container");
    prog_s3::DataList my_list{};
    std::cout << "List contents: " << std::endl;
    prog_s3::StudentData* simple_ptr{0};
    prog_s3::StudentDataDebt* debt_ptr{0};
    prog_s3::StudentDataBudget* budget_ptr{0};
    try {
        simple_ptr = new prog_s3::StudentData("SData", 33, 3.3);
        debt_ptr = new prog_s3::StudentDataDebt("SDataDebt", 66, 66.67, 66);
        budget_ptr = new prog_s3::StudentDataBudget("SDataBudget", 100, 99.99,
9999.99);
    } catch (std::bad_alloc ex) {
        std::cout << "print_polymorphic() - bad alloc on init" << std::endl;
        std::cerr << ex.what() << std::endl;
        if (simple_ptr) delete simple_ptr;
        if (debt_ptr) delete debt_ptr;
        if (budget_ptr) delete budget_ptr;
        print_closing("Simple StudentData container end");
        return;
    }

    std::cout << "Expected pos: 0 ";
    simple_ptr->print_data();
    std::cout << "Expected pos: 1 ";
    debt_ptr->print_data();
    std::cout << "Expected pos: 2 ";
    budget_ptr->print_data();
    /*
    push_front просто будет для процессора дешевле,
    так что сделаю им, но в обратном порядке
    */
    try {
        my_list.push_front(*budget_ptr);
        my_list.push_front(*debt_ptr);
        my_list.push_front(*simple_ptr);
    } catch (std::bad_alloc ex) {
        std::cout << "print_polymorphic() - bad alloc on list.push_front()" <<
std::endl;
        std::cerr << ex.what() << std::endl;
        if (simple_ptr) delete simple_ptr;
        if (debt_ptr) delete debt_ptr;
        if (budget_ptr) delete budget_ptr;
        print_closing("Simple StudentData container end");
        return;
    }

    //они нам просто больше не нужны
    if (simple_ptr) delete simple_ptr;
    if (debt_ptr) delete debt_ptr;
    if (budget_ptr) delete budget_ptr;

```



```

std::cout << "List structure: " << std::endl;
my_list.print_structure();

print_title("Polymorphic behaviour example");
try {
    std::cout << "my_list.get(0)->print_data(): " << std::endl;
    my_list.get(0)->print_data();
    std::cout << "my_list.get(1)->print_data(): " << std::endl;
    my_list.get(1)->print_data();
    std::cout << "my_list.get(2)->print_data(): " << std::endl;
    my_list.get(2)->print_data();
} catch (std::out_of_range ex) {
    std::cout << "print_polymorphic() - out of range on list.get()" <<
std::endl;
    std::cerr << ex.what() << std::endl;
}

print_closing("Polymorphism example end");
print_closing("Simple StudentData container end");
}

void print_exception() {
    print_title("Exceptions example");
    prog_s3::DataList list;
    std::cout << "Trying: DataList{}.get(64): ";
    try {
        list.get(64);
        std::cout << "Success, somehow" << std::endl;
    } catch (std::out_of_range ex) {
        std::cout << "std::out_of_range caught!" << std::endl;
        std::cerr << "\t" << ex.what() << std::endl;
    }

    std::cout << "Trying: DataList{}.remove_at(0): ";
    try {
        list.remove_at(0);
        std::cout << "Removed nothing, what a miracle" << std::endl;
    } catch (std::out_of_range ex) {
        std::cout << "std::out_of_range caught!" << std::endl;
        std::cerr << "\t" << ex.what() << std::endl;
    }
    print_closing("Exceptions example");
}

int main(void) {
    try {
        print_simple_part();
        print_various_constructors_part();
        print_getters();
        print_prints();
        print_assignment();
        print_increment();
        print_addition();
        print_subtraction();
        print_conversion();
        print_file_io();
        print_derived_constructors();
        print_derived_virtuals();
        print_polymorphic();
        print_exception();
    } catch (std::bad_alloc ex) {
        /*

```

```
        в целом, каждый момент аллокации и так обёрнут в свой,  
        но если мы забыли или не знали - тут хоть поймает.  
    */  
    std::cerr << ex.what() << std::endl;  
    return 1;  
} catch (...) {  
    std::cerr << "unhandled exception" << std::endl;  
    return 2;  
}  
  
return 0;  
}
```

Выводы

Функционал класса списка, использованного в предыдущей лабораторной работе, был изменён подобно стандартным контейнерам так, что попытка получить элемент по несуществующему индексу вызывает исключение. Также, поскольку операторы `new` и `delete` при нехватке памяти и/или иной проблеме при работе с динамической памятью выбрасывают исключение, каждый блок, в котором так или иначе выделяется память был помещён в обработчик `try/catch`. В рамках демонстрационной программы обработка подобного исключения заключается исключительно в прерывании выполнения текущего блока и очистке выделенной памяти (если таковая имеется) поскольку в процессе раскрутки стека память по указателям не очищается автоматически.

Был изучен класс `std::exception` и его наследники, получено представление о логике обработки исключений.